

Develop Thread-Safe Code



Cătălin Tudose

Java Champion, PhD in Computer Science, Java and Web Technologies Expert

www.catalintudose.com | www.linkedin.com/in/catalin-tudose-847667a1



Thread Safety

**Thread safety in concurrent
access**

Unexpected behavior

Race conditions

Data corruption



The Lost Update Problem

**Concurrency
problem**

Uncontrolled way

The same resource

Write–write conflict

Overwritten by data





Demo: Lost Update Problem





Atomic Variables



Synchronizing the Access

Simple primitive
variable

Synchronization
mechanisms

synchronized block

volatile

AtomicInteger/Long/
Boolean

Compare-and-swap





Demo: Atomic Variables





Locks



java.util.concurrent.locks.Lock

Like synchronized

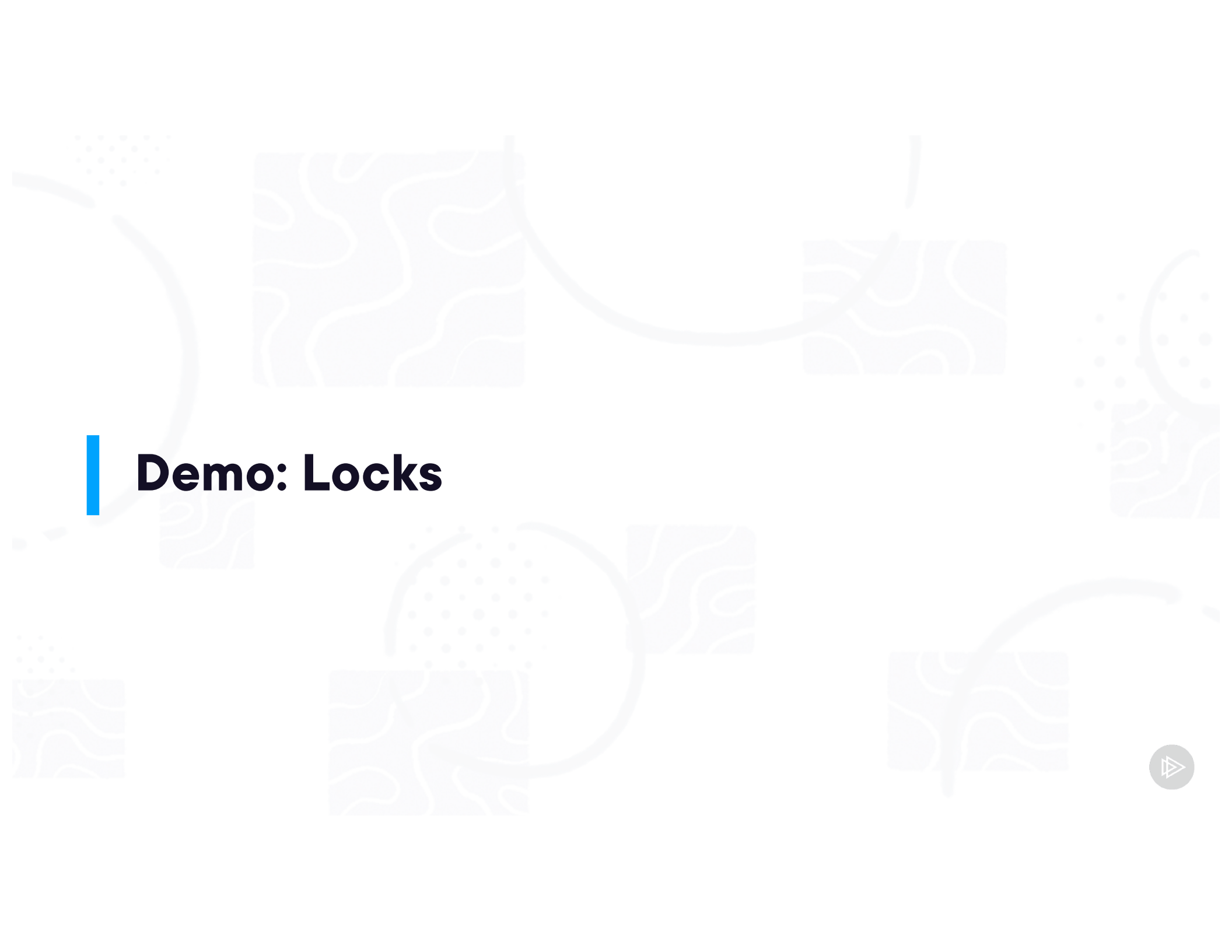
Unlocking

ReentrantLock

tryLock()

**ReentrantReadWrite
Lock**





Demo: Locks

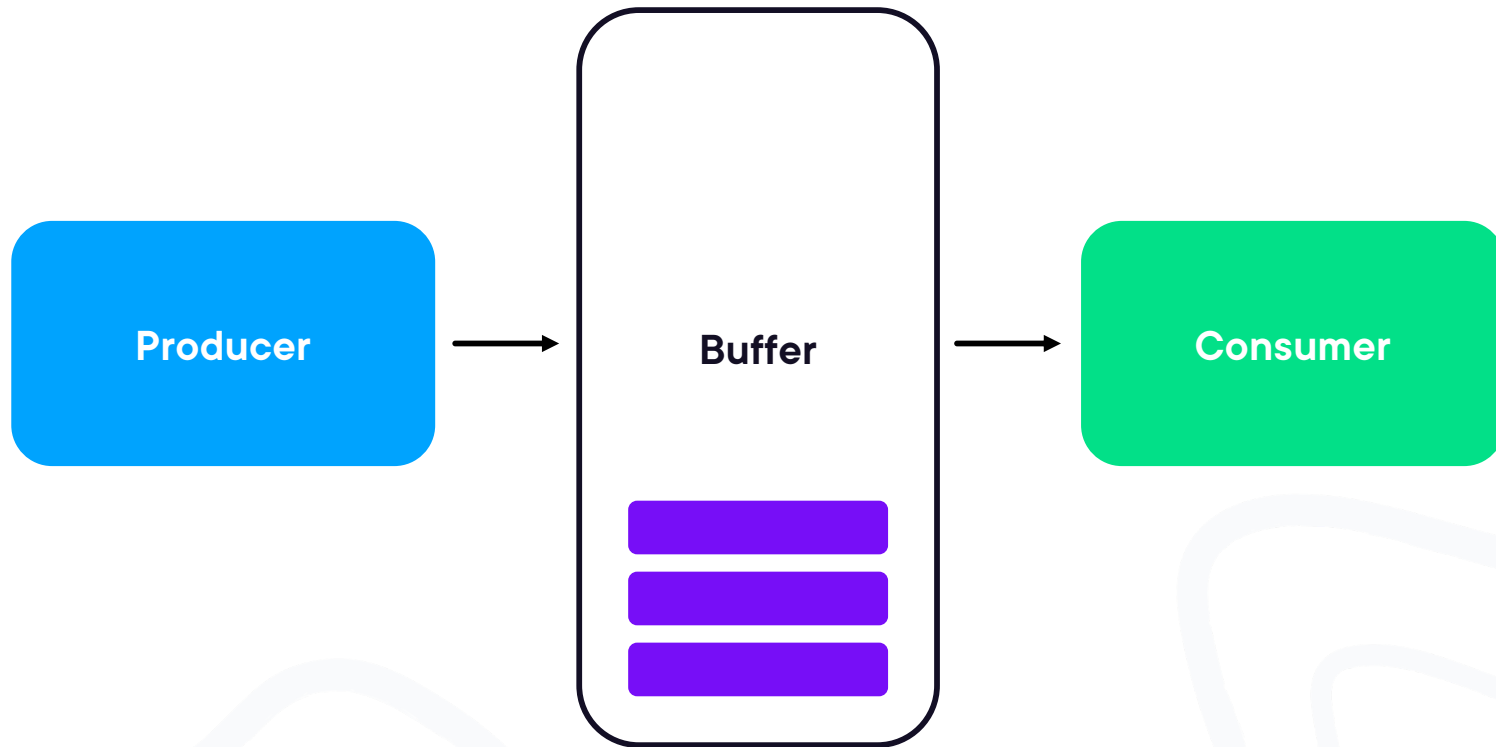




Producer-Consumer Problem



Producer-Consumer Problem



The background features several light gray squares with white wavy patterns, some circles with dots, and thin curved lines.

Demo: Producer-Consumer Problem

